Indian Institute of Technology Kharagpur

CS39006 – Database Management Systems Laboratory

Project Proposal

Hybrid Trading Database System

High-QPS Order Matching, Temporal Analytics,
and Quantum-Assisted Arbitrage Detection

Course — CS39006 – Database Management Systems

Semester — Spring 2026

Submitted — March 11, 2026

Final Demo — April 15, 2026

Group Members

#	Name	Roll Number
1	Ashutosh Sharma	23CS10005
2	Sujal Anil Kaware	23CS30056
3	Parag Mahadeo Chimankar	23CS10049
4	Kshetrimayum Abo	23CS30029
5	Kartik Pandey	23CS30026

Hybrid Trading Database System: High-QPS Order Matching, Temporal Analytics, and Quantum-Assisted Arbitrage Detection

Guideline Coverage: #2 High-QPS Systems · #6 Temporal Databases · #10 Graph Databases · #28 Database Security · #29 Performance Benchmarking

Abstract

This project presents the design and full implementation of a **Hybrid Trading Database System** that integrates five specialized database paradigms into a single, production-inspired infrastructure. The system consists of: (1) a **High-QPS In-Memory Limit Order Book (LOB)** engine built in C++20 with a native `uWebSockets` network layer, `librdkafka` Kafka consumer/producer, and a lock-free LMAX-pattern three-thread pipeline achieving >100,000 order operations per second; (2) a **TimescaleDB Temporal Database** ingesting tick data via `psycopg3` binary COPY with hierarchical OHLCV continuous aggregates (1 min/5 min/15 min/1 h) and pure-SQL live technical indicators (VWAP, SMA-20, RSI-14, Bollinger Bands); (3) an **Apache AGE Graph Database** layer modelling 20 currency/crypto assets as a directed weighted graph with sub-second live edge-weight updates driven by the LOB best-bid and Bellman-Ford arbitrage running every 500 ms; (4) a **Quantum-Assisted Arbitrage Detection Engine** (IBM Qiskit AerSimulator) encoding all $P(N, 3)$ directed 3-cycles as basis states, applying a multi-controlled Grover Oracle and Diffuser, and producing an $O(\sqrt{N})$ vs. $O(N)$ benchmark comparison; and (5) a **Security and Observability Layer** with a `sqlglot` AST-based SQL injection firewall, a Redis sliding-window rate limiter at 1,000 req/s/IP, and a five-panel live Grafana dashboard fed by Prometheus.

System Architecture Overview

▶ Data-Flow Pipeline

[LAYER 0] `Market Data Ingestion` Binance/Alpha Vantage WebSocket → Kafka `raw_orders` (4 partitions)

[LAYER 1] `High-QPS LOB Engine (Module 1)` C++20 · uWebSockets · librdkafka · RB-Tree + FIFO · 3-thread pipeline · >100k ops/s

[LAYER 2] `TimescaleDB Temporal Store (Module 2)` Hypertable `raw_ticks` · 4 OHLCV aggs · VWAP / SMA-20 / RSI-14 / Bollinger

[LAYER 3] `Graph Layer -- Apache AGE (Module 3)` 20 Asset nodes · EXCHANGE edges · Bellman-Ford every 500ms (PRIMARY)

[LAYER 4] `Quantum Arbitrage Engine (Module 4)` Grover Oracle + Diffuser (Qiskit Aer) · $O(\sqrt{N})$ benchmark only · every 10s

[LAYER 5] `Security, Monitoring & Benchmarking (Module 5)` sqlglot AST firewall · Redis rate limiter · Grafana 5-panel · Siege DDoS

▶ Architecture Decision Record – Bellman-Ford vs. Quantum

ADR-001 — Primary Arbitrage Algorithm (Adopted March 9, 2026)

Decision: Bellman-Ford is the *production* arbitrage algorithm. Grover's Algorithm is a *research benchmark* only.

Criterion	Bellman-Ford	Grover (AerSimulator)
Correctness	100% deterministic	50–85% probabilistic
Speed at $N = 20$	< 5 ms	~10,000 ms
Finds all cycles	Yes	No (most-probable only)
Production viable	**Yes**	No (state-vector overhead)
Academic value	Graph algorithms	Complexity-theory proof

Implementation rule: Bellman-Ford runs every 500 ms and writes `method='CLASSICAL'` rows. Grover runs every 10 s and writes `method='QUANTUM'` rows. Both land in the same `arbitrage_signals` hypertable; Grafana Panel 4 displays both streams colour-coded.

▶ Docker Compose Service Graph

All services are orchestrated in a single `docker-compose.yml`. The dependency chain is:

```
zookeeper → kafka → {lob-engine :8001, data-ingestor :8002}
postgres → {quantum-engine :8004, lob-engine}
{postgres, redis, lob-engine, quantum-engine} → fastapi-proxy :8000
fastapi-proxy → prometheus :9090 → grafana :3000
```

Known docker-compose bugs resolved in Phase 0: missing `REDIS_ADDR` on `redis-exporter`; missing `?sslmode=disable` on `postgres-exporter`; missing `env_file: .env` on four services.

Module Descriptions

Module 1 — High-QPS Limit Order Book Matching Engine

– **Engine Source and Build**

The matching core is an external Git submodule cloned from `https://github.com/ashuwhy/lob` into `module1_lob/engine/`. The rule is: wrap it with a high-performance C++ network layer. Key source files:

File	Purpose
`cpp/src/book_core.cpp`	Matching engine – Red-Black Tree + FIFO queues per price level
`cpp/include/lob/mempo`	Arena allocator / lock-free memory pool ($O(1)$ alloc)
`cpp/src/replay.cpp`	TAQ event replay
`python/olob/_bindings`	pybind11 bridge → Python `olob` module (legacy; replaced by native C++)

– Core Data Structures

Structure	Role	Complexity
Red-Black Tree	Price-level index, bid and ask sides	`O(log M) insert/del`
Per-price FIFO deque	Price-time priority matching	`O(1) match / cancel`
`mempool.hpp` arena	Lock-free order allocation	`O(1) alloc`
Lock-free Ring Buffer	Inter-thread event passing	`Lock-free`

– C++20 Three-Thread Pipeline (LMAX Disruptor Pattern)

Listing 1: Concurrency model – three dedicated threads with lock-free queues

```cpp
// Thread A -- InboundThread
//   librdkafka Consumer on 'raw_orders' topic (4 partitions)
//   Parses Kafka message JSON -> Order struct (zero-copy simdjson)
//   Pushes Order -> inbound_queue (lock-free ring buffer)

// Thread B -- MatchingThread
//   Pops Order from inbound_queue
//   Calls C++ OrderBook book.place(order)  -- nanosecond match
//   Pushes resulting Trades -> outbound_queue
//   Broadcasts depth-diff frames via uWebSockets to all subscribers

// Thread C -- OutboundThread (Persistence)
//   Pops TradeEvents from outbound_queue
//   librdkafka Producer -> 'executed_trades' topic
//   Module 2 data-ingestor consumes this topic & bulk-COPYs to raw_ticks
```

– Network Layer – C++ uWebSockets (Tier 1 – HFT Pivot)

Real HFT exchanges (Binance, OKX) serve packets directly from C++ memory. By replacing the earlier Python FastAPI tier with a native C++ `uWebSockets` server, the per-request overhead drops from ∼100 µs to <1 µs:

Listing 2: uWebSockets REST route calling the C++ OrderBook directly

```cpp
#include "uWebSockets/App.h"
#include "lob/book_core.hpp"

```

```cpp
int main() {
    OrderBook book;
    uWS::App().post("/lob/order", [&book](auto *res, auto *req) {
        // zero-copy JSON parse with simdjson
        Order o = parse_order(req->getBody());
        auto trades = book.place(o);          // < 1 us match
        res->writeStatus("201 Created")
           ->end(to_json(trades));
    })
    .ws<DepthData>("/lob/stream/:symbol", {
        .message = [&book](auto *ws, std::string_view msg, uWS::
            OpCode) {
            // publish depth-diff frames on every match
        }
    })
    .listen(8001, [](auto *token) {})
    .run();
}
```

Tier	Approach	Est. QPS	Status
1	C++ uWebSockets – packets arrive in C++ memory	100,000+	**Implementing**
2	Persistent WebSocket + binary msgpack frames	10,000+	Future optimisation
3	Python FastAPI + orjson	5,000+	Deprecated

– REST + WebSocket API Endpoints

Method	Path	Action
POST	/lob/order	Add limit or market order → HTTP 201 + {order_id, status, filled_qty}
DELETE	/lob/order/{id}	Cancel an open order
PATCH	/lob/order/{id}	Atomic cancel + re-place at new price/qty
GET	/lob/depth/{symbol}	Top-10 bid/ask levels as JSON
WS	/lob/stream/{symbol}	Async depth-diff broadcast per match event
GET	/lob/metrics	Prometheus exposition (prometheus-cpp)

– Prometheus Metrics (prometheus-cpp)

Listing 3: Native C++ Prometheus counters and histograms

```cpp
Family<Counter>&   lob_orders_total;    // labels: {symbol, side}
Family<Counter>&   lob_trades_total;    // labels: {symbol}
Family<Histogram>& lob_order_latency;  // latency inside matching
    engine (ns)
Family<Gauge>&     lob_active_orders;  // labels: {symbol}
```

– Benchmarking Target

Listing 4: Siege stress test – 200 concurrent users for 30 seconds

```bash
siege -c 200 -t 30S --content-type "application/json" \
```

```
      -f module1_lob/urls.txt
```

Target: >100,000 order ops/sec; p99 latency ≤10 ms. Nanosecond-level baseline already recorded in `bench_out/latencies.csv` and `bench_out/latency_histogram.png` from the submodule.

Module 2 — TimescaleDB Temporal Analytics Engine

– Ingestion Pipeline

Trade executions flow from Module 1 via Kafka topic `executed_trades` (not `raw_orders`). The `kafka_consumer.py` process batches 1,000 records or 100 ms timeout, whichever comes first, and flushes via `psycopg3` binary `COPY` into the `raw_ticks` hypertable. A synthetic data generator (`gen_ticks.py`) pre-populates 1 M rows using a Geometric Brownian Motion (GBM) model:

$$dS = S\,(\mu\,dt + \sigma\,dW), \quad \mu = 0,\ \sigma = 0.02$$

across 10 symbols to satisfy the ≥1 M row benchmark baseline.

– Hypertable Schema and Policies

Listing 5: raw_ticks hypertable with space + time partitioning

```sql
CREATE TABLE raw_ticks (
    ts        TIMESTAMPTZ     NOT NULL,
    symbol    VARCHAR(20)     NOT NULL,
    price     NUMERIC(18,8)   NOT NULL,
    volume    NUMERIC(18,8)   NOT NULL,
    side      CHAR(1)         NOT NULL CHECK (side IN ('B','S')),
    order_id  UUID            NOT NULL,
    trade_id  UUID            NOT NULL,
    exchange  VARCHAR(30)     DEFAULT 'SYNTHETIC'
);

-- Hypertable: 1-day chunks, 4 space partitions by symbol hash
SELECT create_hypertable('raw_ticks', 'ts',
    partitioning_column => 'symbol',
    number_partitions   => 4,
    chunk_time_interval => INTERVAL '1 day');

-- Compression after 7 days; retention drop after 90 days
ALTER TABLE raw_ticks SET (
    timescaledb.compress,
    timescaledb.compress_segmentby = 'symbol',
    timescaledb.compress_orderby   = 'ts DESC');
SELECT add_compression_policy('raw_ticks', INTERVAL '7 days');
SELECT add_retention_policy  ('raw_ticks', INTERVAL '90 days');
```

– Four Continuous Aggregates (OHLCV)

All four materialized views are defined in `init.sql` inside idempotent `DO $$ IF NOT EXISTS $$` blocks. The 1-minute aggregate is the base; coarser granularities cascade from it:

Listing 6: 1-minute OHLCV continuous aggregate (representative – 5m/15m/1h follow same pattern)

```sql
CREATE MATERIALIZED VIEW ohlcv_1m
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 minute', ts) AS bucket,  symbol,
    FIRST(price, ts)  AS open,   MAX(price)  AS high,
    MIN(price)        AS low,    LAST(price, ts) AS close,
    SUM(volume)       AS volume
FROM raw_ticks
GROUP BY bucket, symbol
WITH NO DATA;

SELECT add_continuous_aggregate_policy('ohlcv_1m',
    start_offset      => INTERVAL '10 minutes',
    end_offset        => INTERVAL '1 minute',
    schedule_interval => INTERVAL '1 minute');
```

View	Bucket	Schedule	Start Offset	End Offset
ohlcv_1m	1 min	1 min	10 min	1 min
ohlcv_5m	5 min	5 min	30 min	5 min
ohlcv_15m	15 min	15 min	1 hour	15 min
ohlcv_1h	1 hour	1 hour	4 hours	1 hour

– SQL Technical Indicators

All four indicators are implemented as pure SQL functions or inline window queries – no Python computation:

Listing 7: VWAP and SMA-20 as SQL stored functions

```sql
-- VWAP: Volume Weighted Average Price over a time window
CREATE OR REPLACE FUNCTION fn_vwap(
    p_symbol TEXT, p_from TIMESTAMPTZ, p_to TIMESTAMPTZ)
RETURNS NUMERIC AS $$
  SELECT SUM(price * volume) / NULLIF(SUM(volume), 0)
  FROM raw_ticks
  WHERE symbol = p_symbol AND ts BETWEEN p_from AND p_to;
$$ LANGUAGE sql STABLE;

-- SMA-20: 20-period Simple Moving Average using the last 20 1m
    candles
CREATE OR REPLACE FUNCTION fn_sma20(p_symbol TEXT, p_at TIMESTAMPTZ)
RETURNS NUMERIC AS $$
  SELECT AVG(close) FROM (
    SELECT close FROM ohlcv_1m
    WHERE symbol = p_symbol AND bucket <= p_at
    ORDER BY bucket DESC LIMIT 20) t;
$$ LANGUAGE sql STABLE;
```

Listing 8: RSI-14 using LAG window functions (pure SQL, no UDF dependency)

```sql
WITH gains_losses AS (
  SELECT bucket, symbol, close,
    GREATEST(close - LAG(close) OVER
```

```
        (PARTITION BY symbol ORDER BY bucket), 0)  AS gain,
    GREATEST(LAG(close) OVER
        (PARTITION BY symbol ORDER BY bucket) - close, 0) AS loss
  FROM ohlcv_1m
),
avg_gl AS (
  SELECT bucket, symbol, close,
    AVG(gain) OVER (PARTITION BY symbol ORDER BY bucket
        ROWS BETWEEN 13 PRECEDING AND CURRENT ROW)  AS avg_gain,
    AVG(loss) OVER (PARTITION BY symbol ORDER BY bucket
        ROWS BETWEEN 13 PRECEDING AND CURRENT ROW)  AS avg_loss
  FROM gains_losses
)
SELECT bucket, symbol,
    100 - (100 / (1 + avg_gain / NULLIF(avg_loss, 0)))  AS rsi_14
FROM avg_gl;
```

– Benchmarking – Hypertable vs. Plain Table

`bench_timescale.py` creates a vanilla `raw_ticks_plain` (same schema, no TimescaleDB extension), loads the same 1 M rows into both, and runs identical OHLCV range queries 10 times on each. Results are written to `benchmark_runs` and exported as `benchmark_timescale.png`. Expected outcome: hypertable is $\geq 10\times$ faster on time-range scans due to chunk-exclusion pruning.

Module 3 — Graph Database Layer + Primary Arbitrage Engine

Architecture note: Bellman-Ford is the *primary production arbitrage algorithm* for this module. The quantum module (Module 4) consumes this module's `GET /graph/rates` output for benchmarking only.

– Graph Schema

The multi-currency market is modelled as a directed weighted graph inside PostgreSQL via the Apache AGE extension (Cypher query language):

- **Nodes:** 20 `Asset` vertices – 10 crypto (BTC ETH BNB SOL ADA XRP DOGE AVAX MATIC DOT) and 10 fiat (USD EUR GBP JPY AUD CAD CHF INR SGD HKD)
- **Edges:** Directed `EXCHANGE` edges for all active trading pairs
- **Edge properties:** `bid FLOAT`, `ask FLOAT`, `spread FLOAT`, `last_updated BIGINT`

Acceptance criteria verified post-init:

Listing 9: Minimum graph population checks

```
SELECT * FROM cypher('fx_graph', $$
    MATCH (n:Asset) RETURN count(n) $$) AS (c agtype);
-- Must return 20

SELECT * FROM cypher('fx_graph', $$
    MATCH ()-[r:EXCHANGE]->() RETURN count(r) $$) AS (c agtype);
-- Must return >= 50
```

– Live Edge-Weight Updater (`edge_weight_updater.py`)

An asyncio loop running every 500 ms polls `GET /lob/depth/{symbol}` for best-bid / best-ask and updates the corresponding AGE edge:

Listing 10: Cypher SET query issued every 500 ms to keep edge weights live

```
SELECT * FROM cypher('fx_graph', $$
    MATCH (a:Asset {symbol: $from})-[r:EXCHANGE]->(b:Asset {symbol:
        $to})
    SET r.bid = $bid, r.ask = $ask, r.last_updated = timestamp()
$$, $1) AS (result agtype);
-- Latency target: edge updated within 600 ms of any LOB price change
```

– Bellman-Ford Arbitrage (Primary – runs every 500 ms)

Negative-log-transformed exchange rates turn the arbitrage problem into a negative-cycle detection problem, which Bellman-Ford solves in $O(V \cdot E)$:

1. `build_rate_matrix(conn)` queries all AGE `EXCHANGE` edges via `psycopg3` and builds an $N \times N$ float adjacency matrix.
2. Transform: $w_{ij} = -\log(r_{ij})$ where r_{ij} is the bid exchange rate.
3. Run Bellman-Ford relaxation over all $V - 1$ passes; a further relaxation identifies any negative-weight cycle, i.e., $\prod r_{ij} > 1$.
4. Write detected cycle path + `profit_pct` to `arbitrage_signals` with `method='CLASSICAL'`.

Listing 11: Cypher path query – find all profitable 3-hop cycles from USD

```
SELECT * FROM cypher('fx_graph', $$
    MATCH p = (a:Asset {symbol: 'USD'})-[*3]->(a)
    WHERE reduce(rate=1.0, r IN relationships(p) | rate * r.bid) >
        1.0
    RETURN p
$$) AS (path agtype);
```

– Graph API Endpoints

Endpoint	Description
`GET /graph/nodes`	List all 20 Asset vertices
`GET /graph/edges`	List EXCHANGE edges with current bid/ask weights
`GET /graph/paths?from_symbol=`	Run Bellman-Ford; return 3-hop cycle results
`GET /graph/rates`	$N \times N$ adjacency matrix JSON (consumed by Module 4)
`GET /graph/health`	Node count + last edge-update timestamp

Module 4 — Quantum Arbitrage Detection Engine

Scope clarification (ADR-001): Grover's Algorithm is a *research benchmark only.* AerSimulator is slower than Bellman-Ford because it classically simulates the full state vector. The expected and *documented* result is: Bellman-Ford as a near-flat line; Grover rising exponentially. This is the academically strongest result – it demonstrates the theoretical $O(\sqrt{N})$ oracle-query complexity while correctly attributing the simulation overhead.

– State Encoding

All $P(N, 3) = N!/(N-3)!$ directed 3-cycles are enumerated and mapped to basis states of a quantum register:

Listing 12: Qubit register sizing as a function of N

```
n_qubits = math.ceil(math.log2(len(all_cycles)))
# N=8  ->  336 cycles -> 9 qubits
# N=16 -> 3360 cycles -> 12 qubits
# N=32 -> 29760 cycles -> 15 qubits
# Cap at N=64 (beyond this AerSimulator exhausts RAM)
```

– Grover Oracle

The oracle marks every basis state $|i\rangle$ that corresponds to a profitable cycle ($r_{AB} \times r_{BC} \times r_{CA} > 1$) by applying a phase flip of -1:

$$\mathcal{O}\,|i\rangle = \begin{cases} -|i\rangle & \text{if cycle } i \text{ is profitable} \\ |i\rangle & \text{otherwise} \end{cases}$$

Listing 13: Multi-controlled Grover Oracle (phase-kickback via ancilla)

```
def build_oracle(profitable_states: list[int], n_qubits: int) ->
    QuantumCircuit:
    qr = QuantumRegister(n_qubits)
    qc = QuantumCircuit(qr)
    for state in profitable_states:
        binary = format(state, f'0{n_qubits}b')
        # Flip qubits where bit == '0' (pre-conditioning)
        for i, bit in enumerate(reversed(binary)):
            if bit == '0': qc.x(qr[i])
        # Multi-controlled X with phase kickback on last qubit
        qc.h(qr[-1])
        qc.mcx(list(range(n_qubits - 1)), n_qubits - 1)
        qc.h(qr[-1])
        # Undo pre-conditioning
        for i, bit in enumerate(reversed(binary)):
            if bit == '0': qc.x(qr[i])
    return qc
```

– Grover Diffuser

The diffuser amplifies marked states by reflecting the current amplitude vector around the uniform superposition $|s\rangle = H^{\otimes n}|0\rangle^n$. Its unitary is $2|s\rangle\langle s| - I$:

Listing 14: Grover Diffuser (inversion-about-mean)

```
def build_diffuser(n_qubits: int) -> QuantumCircuit:
    qr = QuantumRegister(n_qubits)
    qc = QuantumCircuit(qr)
    qc.h(range(n_qubits))
    qc.x(range(n_qubits))
    qc.h(n_qubits - 1)
    qc.mcx(list(range(n_qubits - 1)), n_qubits - 1)
    qc.h(n_qubits - 1)
    qc.x(range(n_qubits))
    qc.h(range(n_qubits))
    return qc
    # Verify: unitary == 2|s><s| - I for N=2 test case
```

– Full Grover Run

The circuit is initialised in uniform superposition and the oracle + diffuser are applied for $\approx \frac{\pi}{4}\sqrt{N/M}$ iterations (where M is the number of marked states), after which a projective measurement collapses to the most probable profitable cycle:

Listing 15: End-to-end Grover circuit construction and simulation

```
def run_grover(rates_matrix, nodes, shots=1024) -> dict:
    cycles     = enumerate_cycles(nodes, k=3)          # P(N,3) entries
    profitable = [i for i,c in enumerate(cycles)
                     if is_profitable(c, rates_matrix)]
    n_qubits   = math.ceil(math.log2(len(cycles)))
    n_iter     = max(1, int(
        math.pi / 4 * math.sqrt(len(cycles) / max(len(profitable), 1)))
    qc = QuantumCircuit(n_qubits, n_qubits)
    qc.h(range(n_qubits))                              # uniform superposition
    for _ in range(n_iter):
        qc.compose(build_oracle(profitable, n_qubits), inplace=True)
        qc.compose(build_diffuser(n_qubits),           inplace=True)
    qc.measure(range(n_qubits), range(n_qubits))
    result     = AerSimulator().run(qc, shots=shots).result()
    best_idx   = int(max(result.get_counts(),
                         key=result.get_counts().get), 2)
    return {"path": cycles[best_idx],
            "circuit_depth": qc.depth(),
            "grover_iterations": n_iter}
```

– Background Service (`quantum_service.py`)

Listing 16: 10-second background loop – both algorithms on the same rate matrix

```
async def background_loop():
    while True:
        rates = await fetch("/graph/rates")         # live N x N matrix
        t0 = time.perf_counter()
        bf_result = bellman_ford_arbitrage(rates)   # PRIMARY production algo
        classical_ms = (time.perf_counter() - t0) * 1000
```

```python

        t0 = time.perf_counter()
        gr_result = run_grover(rates, nodes)        # RESEARCH benchmark
        quantum_ms = (time.perf_counter() - t0) * 1000

        await db.insert_signal(bf_result, method='CLASSICAL',
                               classical_ms=classical_ms)
        await db.insert_signal(gr_result, method='QUANTUM',
                               quantum_ms=quantum_ms)
        await asyncio.sleep(10)
```

– Benchmark (benchmark_quantum.py)

$N \in \{4, 8, 12, 16, 20, 24, 28, 32\}$ nodes; 10 trials per method per N on a synthetic random rate matrix. Output: `benchmark_quantum.png` – dual line chart, log-scale Y-axis, with $O(N)$ and $O(\sqrt{N})$ reference lines overlaid.

Module 5 — Security, Observability, and DoS Prevention

– Security Proxy Architecture

All external traffic enters exclusively through port 8000. Two middleware layers execute in order before any request reaches a backend service:

`External :8000 → [Rate Limiter] → [SQL AST Firewall] → Proxy Router`

The proxy router dispatches: `/lob/*` → `hqt-lob:8001`; `/analytics/*` and `/graph/*` → routers in the same process; `/quantum/*` → `quantum-engine:8004`.

– SQL Injection Firewall (sql_firewall.py)

Two complementary passes block injection attempts:

Listing 17: Two-pass SQL injection middleware: string scan + sqlglot AST walk

```python
import sqlglot

BANNED = ['DROP','TRUNCATE','UNION SELECT','--','/*',
          'xp_','EXEC','information_schema']

async def sql_injection_middleware(request, call_next):
    payload = (await request.body()).decode() + str(request.query_params)

    # Pass 1: fast string scan (catches direct injection)
    for pattern in BANNED:
        if pattern.upper() in payload.upper():
            await log_event(ip, 'SQL_INJECTION', payload[:500])
            return JSONResponse({"error": "forbidden"}, status_code=403)

    # Pass 2: AST parse via sqlglot (catches obfuscated/encoded payloads)
    try:
        for stmt in sqlglot.parse(payload):
```

```
            if type(stmt).__name__ in ('Drop','Truncate','Create','AlterTable'):
                await log_event(ip, 'SQL_INJECTION', payload[:500])
                return JSONResponse({"error": "forbidden"}, status_code=403)
    except Exception:
        pass  # parse failure = not valid SQL = not a threat

    return await call_next(request)
```

Every blocked request is inserted into the `security_events` hypertable with `event_type='SQL_INJECTION'`, `client_ip`, `raw_payload`, and `endpoint`.

– Redis Sliding-Window Rate Limiter (`rate_limiter.py`)

Listing 18: Per-IP sliding-window counter – 1000 req/s ceiling via Redis INCR+EXPIRE

```
async def rate_limit_middleware(request, call_next):
    ip  = request.client.host
    key = f"rl:{ip}"
    count = await redis_client.incr(key)
    if count == 1:
        await redis_client.expire(key, 1)     # 1-second window
    if count > 1000:
        await log_event(ip, 'RATE_LIMIT', str(request.url))
        return JSONResponse({"error": "rate limit exceeded"}, status_code=429)
    return await call_next(request)
    # Fallback if Redis is down: threading.Semaphore token-bucket
```

– Database Tables for Security Observability

Listing 19: security_events hypertable – indexed by IP and timestamp

```
CREATE TABLE IF NOT EXISTS security_events (
    event_id   UUID        NOT NULL DEFAULT gen_random_uuid(),
    ts         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    client_ip  INET        NOT NULL,
    event_type VARCHAR(30) NOT NULL
               CHECK (event_type IN ('SQL_INJECTION','RATE_LIMIT','AUTH_FAIL')),
    raw_payload TEXT,
    blocked    BOOLEAN     NOT NULL DEFAULT TRUE,
    endpoint   VARCHAR(200),
    PRIMARY KEY (event_id)
);
SELECT create_hypertable('security_events', 'ts');
CREATE INDEX IF NOT EXISTS idx_security_ip_ts ON security_events (client_ip, ts DESC);
```

– Prometheus Metrics

Listing 20: Metrics counters and histograms exposed at /metrics

```
lob_orders_total       = Counter('lob_orders_total', ...,   ['symbol','side'])
```

```
lob_trades_total        = Counter('lob_trades_total', ...,    ['symbol'])
lob_order_latency_ms    = Histogram('lob_order_latency_ms',   ...)
lob_active_orders       = Gauge('lob_active_orders', ...,     ['symbol'])
sql_injections_total    = Counter('security_sql_injections_total', ...)
rate_limit_hits_total   = Counter('security_rate_limit_total', ...)
arb_signals_total       = Counter('quantum_arbitrage_signals_total', ..., ['method'])
```

– Grafana Dashboard – Five Panels

#	Panel type	Source	Query / Data
1	Candlestick	TimescaleDB	`SELECT bucket, open, high, low, close FROM ohlcv_1m WHERE symbol=$symbol`
2	Heatmap	TimescaleDB	LOB depth – bid/ask volume at each price level over time
3	Bar chart	TimescaleDB	`SELECT bucket, SUM(volume) FROM ohlcv_1m GROUP BY bucket ORDER BY bucket DESC LIMIT 60`
4	Table	PostgreSQL	`SELECT ts, path, profit_pct, method FROM arbitrage_signals ORDER BY ts DESC LIMIT 20` – CLASSICAL and QUANTUM rows colour-coded
5	Time series	Prometheus	`rate(lob_orders_total[1m]) + histogram_quantile(0.99, rate(lob_order_latency_ms_bucket[1m]))`

– Siege DDoS Simulation

Listing 21: Siege stress test at 1000 concurrent users for 60 seconds

```
siege -c 1000 -t 60S --log=report/siege_ddos_results.txt \
      --content-type "application/json" -f module1_lob/urls.txt
```

Expected outcome: rate limiter blocks requests above 1,000 req/s/IP; `security_events` accumulates `RATE_LIMIT` rows; LOB `/health` recovers within 2 s after siege ends; Grafana Panel 5 shows the QPS spike and recovery curve.

Database Schema Summary

Schema Info

TimescaleDB hypertables require *single-column* primary keys when those columns are referenced by foreign-key constraints. All tables reside in a single **PostgreSQL 16** instance with two extensions loaded: `timescaledb` for time-series partitioning and `age` for graph storage.

Table	Type	Key Design Points
raw_ticks	Hypertable	1-day chunks; 4 space partitions by symbol; compression after 7d; retention 90d
orders	Regular	Partial index on (symbol, side, price DESC) where status='OPEN'
trades	Hypertable	FK refs orders(order_id); single-col PK (fixed from composite)
ohlcv_1m/5m/15m/1	Cont. Agg	Cascaded from raw_ticks; all 4 required for /analytics/ohlcv
arbitrage_signals	Hypertable	method IN ('CLASSICAL','QUANTUM'); single-col PK; Grover-only cols nullable
security_events	Hypertable	event_type IN ('SQL_INJECTION', 'RATE_LIMIT', 'AUTH_FAIL'); index on (client_ip, ts DESC)
benchmark_runs	Regular	Stores Siege, Python threadpool, TimescaleDB, and quantum benchmark results

Technology Stack

Component	Technology
Data Ingestion	Apache Kafka 7.6 (4 partitions); WebSocket (Binance / Alpha Vantage); confluent_kafka Python consumer
Matching Engine	C++20; uWebSockets network layer; simdjson zero-copy JSON; librdkafka producer/consumer; LMAX Disruptor pattern; prometheus-cpp
Temporal Database	TimescaleDB 2.x (PostgreSQL 16 extension); psycopg3 binary COPY; 4 OHLCV continuous aggregates
Graph Database	Apache AGE (PostgreSQL extension); Cypher query language; Bellman-Ford negative-log transform
Quantum Engine	IBM Qiskit 0.45; Qiskit Aer 0.13 (AerSimulator); Python 3.12; multi-controlled oracle + diffuser
Security Proxy	FastAPI; sqlglot AST parser; Redis 7 sliding-window rate limiter; httpx reverse proxy
Monitoring	Grafana 10.3; Prometheus 2.48; prometheus-cpp; postgres-exporter; redis-exporter; node-exporter
Benchmarking	Siege 4.x (DDoS + QPS); custom Python ThreadPool + ConnectionPool; bench_timescale.py; benchmark_quantum.py
Infrastructure	Docker Compose; PostgreSQL 16 (base for TimescaleDB + AGE); GitHub Actions CI; Debian Bookworm

Goals and Success Criteria

#	Goal	Success Metric
G1	LOB throughput	>100,000 order ops/sec (Siege-verified); p99 $\leq$ 10 ms
G2	TimescaleDB query speed	$\geq$1 M tick records; range queries <50 ms; $\geq$10$\times$ faster than vanilla PostgreSQL
G3	Graph coverage	$\geq$20 currency nodes; $\geq$50 edges; edge weights refresh every 500 ms
G4	Quantum engine	Grover circuit on 16-node graph; $O(\sqrt{N})$ vs. $O(N)$ benchmark plot produced
G5	Security	Blocks all OWASP Top-10 SQL injection payloads; DoS limiter stable under Siege
G6	Observability	Five-panel live Grafana dashboard operational
G7	Report	10-page final report submitted by April 15, 2026

Weekly Work Plan

Timeline	Milestone & Deliverables
Week 1 Mar 8 – Mar 14	**Infrastructure Setup & Proposal Submission** • Submit proposal; freeze all DB schemas in `init.sql`; create GitHub repo with `dev/main` policy • Set up PostgreSQL 16 with TimescaleDB and Apache AGE extensions; verify `CREATE EXTENSION` idempotency • Initialise Kafka broker; create `raw_orders` (4 partitions) and `executed_trades` topics via `kafka-setup` one-shot container • Set up Qiskit 0.45 + Aer 0.13 local environment; verify AerSimulator runs 2-qubit test circuit • Fix all known docker-compose bugs: `redis-exporter` env, `postgres-exporter` SSL, missing `env_file`
Week 2 Mar 15 – Mar 21	**Core Engine Implementation** • C++ LOB: clone submodule; wire `uWebSockets` + `librdkafka` + `prometheus-cpp` into `lob_server`; 3-thread pipeline *(M1)* • Kafka consumer → `psycopg3` binary COPY pipeline; GBM synthetic data generator 1 M rows *(M2)* • Hypertable creation + retention + compression policies; verify via `timescaledb_information.hypertables` *(M2)* • Bellman-Ford classical arbitrage baseline: `build_rate_matrix()` + negative-log transform + cycle detection *(M3)* • Add `GET /health` stub to `quantum_api.py` (unblocks fastapi-proxy startup) *(M4)*

(continued...)

Timeline	Milestone & Deliverables
Week 3 Mar 22 – Mar 28	**Intermediate Evaluation Preparation** • All 4 OHLCV continuous aggregates + SQL indicator functions (`fn_vwap`, `fn_sma20`, RSI-14 window query) *(M2)* • AGE graph: 20-node + 50-edge population via `MERGE`; 500 ms edge-weight updater; acceptance criteria verified *(M3)* • Grover Oracle + Diffuser circuit for 8-node graph; `quantum_service.py` background loop *(M4)* • FastAPI SQL injection middleware (string scan + `sqlglot` AST); Redis rate limiter; `security_events` insert *(M5)* • **Intermediate demo: LOB engine + TimescaleDB pipeline live**
Week 4 Mar 29 – Apr 7	**Quantum Module, Security Integration & Full E2E Test** • Scale Grover circuit to $N \in \{8, 16, 24, 32\}$; run `benchmark_quantum.py`; produce `benchmark_quantum.png` *(M4)* • `arbitrage_signals` table receiving both `CLASSICAL` (500 ms) and `QUANTUM` (10 s) rows *(M4)* • Siege DDoS simulation (1000 concurrent users, 60 s); verify rate-limiter `HTTP 429` response *(M5)* • Prometheus scrape targets all green; Grafana 5-panel dashboard wired to live data sources *(M5)* • Full end-to-end integration test: Kafka → LOB → TimescaleDB → Graph → Quantum → Security proxy
Week 5 Apr 8 – Apr 14	**Benchmarking, Report & Demo Preparation** • Full Siege QPS benchmark suite; collect `bench_out/latency_histogram.png` and Grafana screenshots • Run `bench_timescale.py`; confirm hypertable ≥10× speedup; export `benchmark_timescale.png` • Write complete 10-page final report (all members); include architecture diagrams, SQL listings, circuit diagrams, and all benchmark plots • Prepare live demo script covering all 5 modules; code freeze; final review and submission
April 15	**Final Submission & Live Demonstration**

Expected Deliverables and Outcomes

1. **LOB Matching Engine:** C++20 binary (`lob_server`) with `uWebSockets` + `librdkafka` + `prometheus-cpp`, sustaining >100,000 order ops/sec at p99 ≤ 10 ms, benchmarked with Siege.

2. **TimescaleDB Analytics Layer:** Hypertable storing ≥1 M synthetic (GBM) tick records; four OHLCV continuous aggregates; pure-SQL VWAP, SMA-20, RSI-14, and Bollinger-Band indicator functions; ≥10× speedup over vanilla PostgreSQL verified by `bench_timescale.py`.

3. **Live Currency Graph:** Apache AGE graph with 20 nodes and ≥50 directed edges; real-time edge weights updated every 500 ms from LOB best-bid/ask; Bellman-Ford arbitrage cycle detection running every 500 ms.

4. **Quantum Arbitrage Engine:** Grover Oracle + Diffuser circuit (Qiskit AerSimulator) for $N \in \{8, 16, 24, 32\}$; `benchmark_quantum.png` demonstrating $O(\sqrt{N})$ theoretical oracle complexity vs. Bellman-Ford; documented AerSimulator simulation overhead.

5. **Security & Monitoring:** Two-pass SQL injection firewall (string + AST) blocking OWASP payloads; Redis rate limiter demonstrated under 1,000-concurrent-user Siege; live 5-panel Grafana dashboard with candlestick, depth heatmap, volume, arbitrage signals (both methods), and QPS/p99 panels.

6. **10-Page Final Report:** Architecture diagrams, all SQL DDL listings, Grover circuit diagrams, and full benchmark results from all five modules.

References

[1] M. Thompson et al., *Disruptor: High performance alternative to bounded queues*, LMAX Exchange, 2011.

[2] M. Fowler, *The LMAX Architecture*, martinfowler.com, 2011.

[3] TimescaleDB Documentation, *Analyse Financial Tick Data with TimescaleDB*.

[4] TraderMade, *Real-Time Market Data to OHLC Candles with TimescaleDB*, 2021.

[5] Classiq, *Grover's Algorithm – Theory and Implementation*, 2024.

[6] L. K. Grover, *A fast quantum mechanical algorithm for database search*, ACM STOC, 1996.

[7] Apache Software Foundation, *Apache AGE – Graph Extension for PostgreSQL*.

[8] IBM Quantum, *Quantum Approximate Optimization Algorithm*, IBM Quantum Docs, 2025.

[9] J. Fulmer, *Siege – HTTP Load Testing and Benchmarking Utility*.

[10] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*.

[11] uNetworking AB, *uWebSockets – Simple, secure and standards compliant sockets*.

[12] Magnus Edenhill, *librdkafka – The Apache Kafka C/C++ library*, GitHub.